

Rapport de projet – RPG en Rust

Auteur : Théo AVRIL

Titre : Développement d'un RPG 2D en Rust avec Macroquad

Date : 23 novembre 2025

Table des matières

1. Contexte de l'histoire du jeu
 2. Explication des différents modules
 3. Répartition du travail
 4. Warnings et qualité de compilation
 5. Fonctionnalités Rust et bibliothèques externes utilisées
-

1. Contexte de l'histoire du jeu

Le jeu développé est un petit RPG 2D dont l'objectif principal est simple : éliminer tous les ennemis présents dans le monde.

Le joueur commence dans une plaine. Pour progresser efficacement, il doit :

1. **Explorer la maison :**
 - Le joueur peut entrer dans une maison via un portail depuis la plaine.
 - À l'intérieur, il doit trouver un coffre qui contient une amélioration d'attaque.
 - Cette amélioration permet de **doubler la valeur d'attaque** du personnage, ce qui rend les combats plus faciles et plus rapides.
2. **Revenir dans la plaine et suivre le chemin :**
 - Une fois renforcé, le joueur revient dans le monde principal.
 - Il suit le chemin vers la droite, ce qui l'amène progressivement vers des zones plus dangereuses.
3. **Vaincre tous les ennemis et ouvrir les coffres :**
 - Dans la plaine et dans les mondes de type spirale, des ennemis se déplacent et attaquent le joueur.
 - Le but est de **tuer tous les ennemis** de ces zones.
 - Certains coffres contiennent du butin ou des messages, mais un coffre particulier permet de **déclencher la victoire**.
4. **Fin du jeu :**
 - Lorsque tous les ennemis ont été éliminés et que le joueur a ouvert le coffre final, un écran de fin s'affiche.
 - Cet écran propose simplement de quitter le jeu (sans possibilité de recommencer une nouvelle partie depuis cet écran).

L'histoire reste volontairement minimale : l'accent est mis sur l'exploration, la gestion du personnage, la compréhension des mécaniques, et le respect des contraintes techniques imposées (modularité, concurrence, gestion sûre des erreurs, etc.).

2. Explication des différents modules

Le projet est structuré de manière modulaire afin de séparer clairement les responsabilités et de faciliter la lecture du code.

2.1. Fichiers de base

- **main.rs**
Point d'entrée du programme. Il initialise la fenêtre Macroquad, crée une instance de **Game** et lance la boucle principale du jeu.
- **types.rs**
Regroupe plusieurs types utilitaires : positions (**Position**), états de combat, messages à afficher à l'écran, etc. Ces types sont partagés entre plusieurs modules.
- **world.rs**
Représente le **monde logique** :
 - grille de tuiles traversables ou non,
 - liste des joueurs,
 - liste des ennemis,
 - méthodes pour faire se déplacer les ennemis, vérifier les collisions, etc.
- **joueur.rs**
Contient la structure du joueur, ses statistiques (vie, attaque, défense, etc.) et les méthodes associées (subir des dégâts, vérifier s'il est en vie, etc.).
- **ennemi.rs**
Décrit la structure des ennemis (position, points de vie, comportement de base) et fournit les fonctions pour les initialiser et les manipuler.

2.2. Module **game** (logique de haut niveau)

Le module **game** regroupe la logique de haut niveau du jeu. Le fichier **game.rs** contient la structure **Game**, qui centralise :

- l'état courant (menu, en jeu, combat, écran de fin),
- les ressources graphiques (textures, police),
- le monde partagé (**World**) protégé par un **Arc<Mutex<...>>**,
- les informations de déplacement et d'animation du joueur,
- les messages à afficher à l'écran,
- les caches d'ennemis pour chaque monde,
- la gestion des coffres.

Autour de ce fichier central, la logique est éclatée dans plusieurs sous-modules spécialisés :

- **game/rendering.rs**
Gère tout le **rendu graphique** :
 - dessin de la carte (tuiles d'herbe, chemin, eau, maison, portails...),
 - dessin du joueur et des ennemis,
 - affichage des barres de vie (via le module **healthbar**),
 - affichage des messages et du HUD, notamment le **compteur d'ennemis restants** dans le coin supérieur droit de l'écran.
- **game/movement.rs**
Contient la logique de **déplacement du joueur**, la gestion des entrées clavier, et la détection des interactions :
 - transitions entre mondes via les portails,
 - lancement des combats lors des collisions avec les ennemis,
 - ouverture des coffres présents sur la carte.
- **game/world_mgmt.rs**
Regroupe la **gestion des mondes** :
 - chargement des différentes cartes (**WorldKind**) et de leurs tuiles,
 - stockage et restauration des ennemis pour chaque monde dans un cache,
 - repositionnement du joueur lors des changements de monde (plaine, maison, spirales),

- préchargement des mondes au lancement pour connaître à l’avance le nombre total d’ennemis,
- déclenchement de la victoire via `trigger_victory`.
- `game/map_utils.rs`
Fournit des **fonctions utilitaires liées aux cartes** :
 - lecture des fichiers `.map` et conversion en grille de `TileType`,
 - génération d’une carte par défaut si les fichiers sont manquants ou invalides,
 - détection des ancres de maisons pour l’affichage,
 - construction de la carte des cases traversables,
 - génération aléatoire des ennemis sur les cases valides,
 - démarrage d’un **thread séparé** qui fait errer les ennemis dans le monde partagé.
- `game/combat.rs`
Regroupe la logique de **combat** :
 - calcul des dégâts,
 - alternance joueur / ennemi,
 - vérification de la mort des combattants,
 - transition entre l’état “exploration” et l’état “combat”.
- `game/coffre.rs`
Gère l’ensemble du **système de coffres** :
 - coffres classiques (butin, messages),
 - coffre spécial de la maison qui confère le **bonus d’attaque** au joueur,
 - coffre de victoire qui déclenche l’écran de fin.
- `game/healthbar.rs`
Regroupe les fonctions de dessin des **barres de vie** du joueur et des ennemis.

Globalement, ce découpage assure une bonne **modularité** : chaque fichier se concentre sur une responsabilité claire, et la structure `Game` joue le rôle de chef d’orchestre.

3. Répartition du travail

Ce projet est une **réalisation individuelle**.

- **Auteur** : Théo AVRIL
- **Contributions externes** : aucune contribution directe d’autres personnes dans le code source.
- **Aides éventuelles** : utilisation de la documentation officielle de Rust, de la documentation de Macroquad et d’outils d’assistance type complétion/IDE, GitHub Copilot avec CHatGPT 5.1, mais toutes les décisions d’architecture, d’organisation des modules et d’implémentation des fonctionnalités ont été prises et intégrées par moi-même.

4. Warnings et qualité de compilation

Le projet est compilé avec `cargo` en mode développement. Au moment de la rédaction de ce rapport :

- La commande :
`cargo check`
s’exécute **sans warnings ni erreurs**.
- En particulier :
 - aucun `unwrap()` ou `expect()` n’est utilisé dans le code,
 - les résultats (`Result`) sont traités avec des `match`, `if let` ou des combinators sûrs (`map_err`, `unwrap_or_else`, etc.),

- les options (`Option`) sont traitées explicitement.
-

5. Fonctionnalités Rust et bibliothèques externes utilisées

Ce projet met en œuvre plusieurs fonctionnalités de Rust ainsi que quelques bibliothèques externes. Cette section ne couvre bien sûr pas toute la théorie, mais illustre **comment ces outils sont utilisés concrètement dans ce projet** comme demandé.

5.1. Gestion de la concurrence : `Arc<Mutex<...>>` et `threads`

Pour gérer les ennemis qui errent dans le monde de manière autonome, le projet utilise :

- `std::sync::Arc` : compteur de références atomique qui permet de partager une même instance de `World` entre plusieurs threads,
- `std::sync::Mutex` : verrou mutualisé qui garantit qu'un seul thread accède en écriture à `World` à la fois,
- `std::thread::spawn` : création d'un thread séparé.

Concrètement, dans `game.rs` et `map_utils.rs` :

- le monde est stocké sous la forme `Arc<Mutex<World>>`,
- la fonction `start_enemy_thread` clone cet `Arc` et lance un thread qui fait périodiquement :
 - un `lock()` sur le mutex,
 - appelle une méthode du monde pour faire se déplacer un peu les ennemis,
 - fait une pause avec `thread::sleep`.

Cela permet d'avoir **un comportement concurrent** (les ennemis se déplacent en fond) tout en conservant la **sécurité mémoire** garantie par Rust.

5.2. Gestion des erreurs avec `Result` et `Option`

La lecture des fichiers de cartes (`.map`) illustre bien l'utilisation de `Result` :

- `parse_world_file` retourne `Result<Vec<Vec<TileType>>, String>`,
- en cas d'erreur d'E/S ou de format, la fonction construit un message explicite et le propage,
- `load_tiles_for_world` utilise `unwrap_or_else` pour :
 - soit utiliser le résultat correct,
 - soit se replier sur une fonction `load_first_world_in_dir` qui cherche une carte de secours,
 - et si tout échoue, `default_map_tiles` fournit une carte par défaut traversable.

Pour les options, on retrouve des motifs comme :

- `if let Some(player) = world.players().get(0) { ... }` pour manipuler le premier joueur s'il existe,
- `find_tile_position(... ) -> Option<Position>` qui renvoie `None` si aucune tuile de ce type n'est trouvée.

Cette approche respecte la contrainte “**pas de unwrap**” tout en rendant les erreurs gérables et en évitant les paniques à l'exécution.

5.3. Bibliothèque graphique `macroquad` et logique de grille

5.3.1. Utilisation concrète de `Macroquad` Le projet utilise la bibliothèque **Macroquad** pour toute la partie graphique et l'interface avec l'utilisateur :

- création de la fenêtre et de la boucle principale de jeu,

- chargement des textures (sprites des tuiles, du joueur, des ennemis, etc.),
- dessin des éléments à l'écran (tuiles, personnages, HUD),
- gestion des entrées clavier (déplacement du joueur, validation dans les menus),
- mesure du temps écoulé entre deux frames pour synchroniser les animations.

La boucle principale est asynchrone (via `next_frame().await`) et ressemble conceptuellement à :

1. lire les entrées (Macroquad fournit par exemple `is_key_pressed`, `is_key_down`, etc.),
2. mettre à jour l'état logique du jeu (position du joueur, ennemis, messages, coffres),
3. appeler les fonctions de rendu de `rendering.rs` qui utilisent les primitives de Macroquad (`clear_background`, `draw_texture_ex`, `draw_text`, ...),
4. attendre la frame suivante.

Les textures sont chargées une seule fois au démarrage dans une structure centrale (`GameTextures`), puis réutilisées à chaque frame pour éviter de recharger des fichiers en permanence.

5.3.2. Matrice logique vs rendu graphique Une particularité importante du projet est la **dissociation nette entre la logique de jeu (en grille) et le rendu graphique (en pixels)**.

- La logique repose sur une **matrice de tuiles** :
 - La carte est représentée par `map_tiles: Vec<Vec<TileType>>`.
 - Chaque case de cette matrice correspond à une tuile logique (`Herbe`, `Chemin`, `Eau`, `Maison`, `Portal`, etc.).
 - Les entités (joueur, ennemis, coffres) connaissent leur position en coordonnées de grille (`Position { x, y }`).
- Le rendu graphique convertit ces coordonnées de grille en **positions en pixels** :
 - Dans `rendering.rs`, pour chaque `(x, y)` de la grille, on calcule une position en pixels via quelque chose du genre `screen_x = x * TILE_SIZE`, `screen_y = y * TILE_SIZE`.
 - On choisit ensuite la bonne texture Macroquad à dessiner en fonction du `TileType` stocké dans la matrice.
 - Le joueur et les ennemis sont également dessinés en fonction de leurs coordonnées de grille, multipliées par la taille de tuile.

Ce découplage apporte plusieurs avantages :

- La **logique de jeu** (collisions, déplacements, vérification des cases traversables) ne dépend pas de la résolution de l'écran ni des détails graphiques.
- La **partie graphique** peut évoluer (changer la taille des tuiles, remplacer les sprites) sans toucher à la logique de déplacement et de collision.
- La carte logique peut être manipulée facilement (chargée depuis un fichier `.map`, copiée, parcourue) grâce à la représentation en `Vec<Vec<TileType>>`.

Ainsi :

- le module `world` et `map_utils` manipulent surtout la **matrice logique** et les coordonnées en grille,
- le module `rendering` est responsable de la **traduction grille → pixels** et des appels à Macroquad pour dessiner le résultat.

5.4. Génération aléatoire avec `rand`

La bibliothèque `rand` est utilisée pour :

- choisir des variantes de tuiles d'herbe ou de chemin (`thread_rng().gen_range(...)`),
- placer aléatoirement les ennemis sur des cases valides de la carte.

Dans `map_utils.rs`, un `thread_rng()` est créé pour chaque génération, ce qui fournit un générateur local au thread courant, simple à utiliser et suffisant pour ce type de jeu.

5.5. Itérateurs et closures

Le code Rust du projet utilise abondamment les **itérateurs** et les **closures**, par exemple :

- pour transformer des collections :

```
tiles.iter()
    .map(|row| {
        row.iter()
            .map(|tile| /* ... */)
            .collect::<Vec<bool>>()
    })
    .collect()
```

- pour filtrer des entrées de répertoires lors du chargement des fichiers `.map`,
- pour calculer les positions des ennemis ou des tuiles d'un certain type.

Cela permet d'écrire un code concis, expressif et sûr, tout en respectant les **bonnes pratiques idiomatiques de Rust**.

En conclusion, ce projet illustre :

- un **petit jeu complet** avec une boucle de gameplay claire (exploration, amélioration, combat, victoire),
- une **architecture modulaire** bien découpée,
- l'utilisation de la **concurrence sûre** grâce à `Arc<Mutex<World>>` et à un thread d'ennemis,
- une **gestion propre des erreurs** sans `unwrap`,
- et l'intégration cohérente de bibliothèques externes comme **Macroquad** et **rand**.